M2 EIF · INFORMATIQUE QUANTITATIVE

Pricer d’options par arbres recombinants

Binomial, trinomial, dividendes discrets, pruning, méthode BBS et grecques, en Python

Doryan Lochet & Cloé Legrand · octobre 2026

I. FONCTIONNEMENT DU PRICER

Trois étapes, tout se pilote depuis Excel

01

Remplir l'Excel

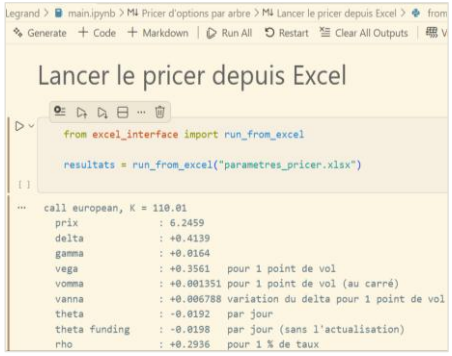
onglet Paramètres, puis enregistrer



02

Exécuter le main

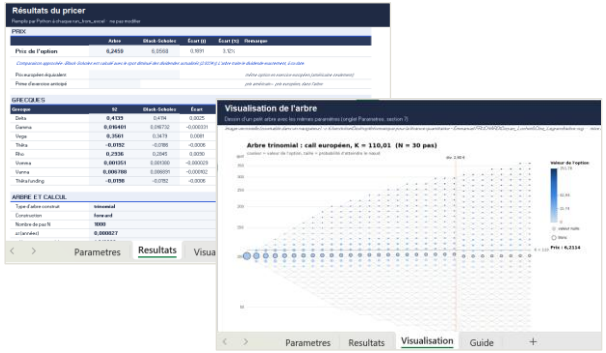
run_from_excel lit le classeur fermé



03

Lire les résultats

onglets Résultats et Visualisation



openpyxl · notre choix

+ Aucune installation : tout le monde peut l'exécuter
− Il faut lancer le main, Excel fermé

xlwings · écarté

+ Bouton dans Excel : tout reste dans le classeur
− Figeait Excel sur nos deux PC

II. VALIDATION DU PRICER

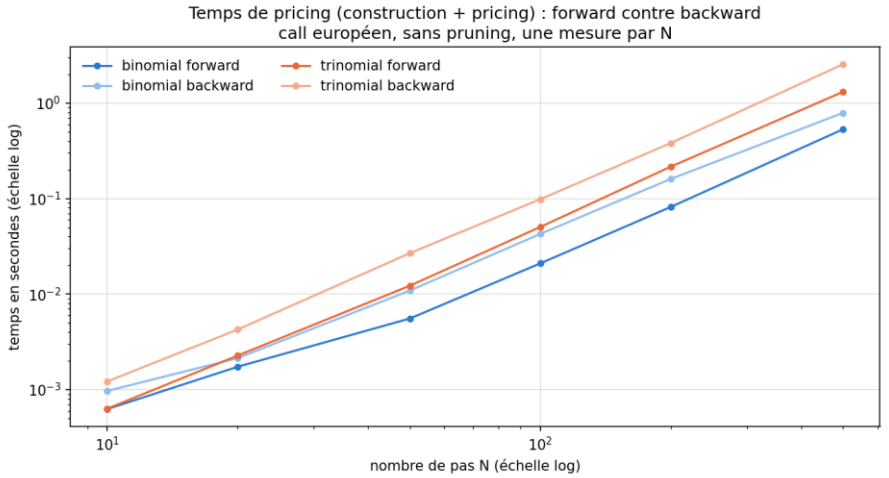
Propriétés financières de l’arbre : 5 tests unitaires

	PROPRIÉTÉ TESTÉE	INTUITION	RÉSULTAT DE L’ARBRE
✓	**Call de strike 0**	Acheter pour 0 = posséder l’action	**100,0000 = S_0**
✓	**Parité call-put**	C − P = S − K·e^(−rT)	**2,9554 = 2,9554**
✓	**Convergence vers Black-Scholes**	Même modèle : plus de pas, même prix	**écart de 0,0002 € à N = 1 000**
✓	**Put américain ≥ européen**	Le droit d’exercer tôt a une valeur	**14,54 ≥ 13,97 (prime 0,57)**
✓	**Dividende de 2 €**	Le spot chute : call ↓, put ↑	**call 11,37 → 10,29 · put 8,42 → 9,31**

Pour converger vers **Black-Scholes** il faut **un grand N**, ce qui augmente le **temps de calcul** et la **mémoire**…

Les résultats empiriques

FORWARD

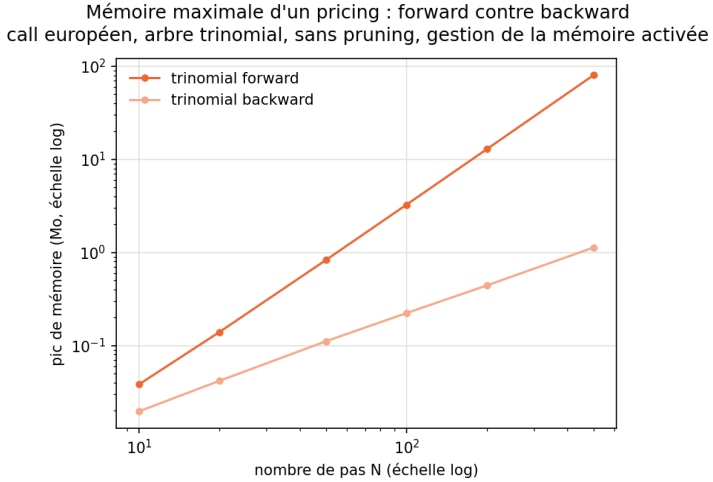


Courbe de temps. Le Forward prend moins de temps de calcul quel que soit *N*.

Plus rapide.

VS

BACKWARD



Mesure *tracemalloc* avec *cut_column* active. Forward en $O(N^2)$, Backward en $O(N)$.

Plus économe en mémoire.

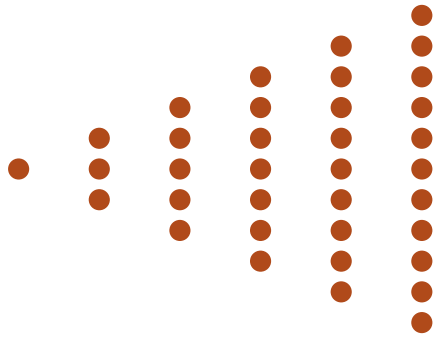
Ce compromis ne vient pas du prunning (les deux tests sont faits sans prunning), mais de la structure même des **deux algorithmes de parcours**.

Pourquoi la méthode Backward divise la mémoire mais multiplie le temps de calcul

FORWARD - Arbre complet conservé

0→T

Tree.build() génère **la totalité du réseau** : chaque *Node* est instancié une seule fois, mais le pic de mémoire en ***O(N²)*** **est atteint**.



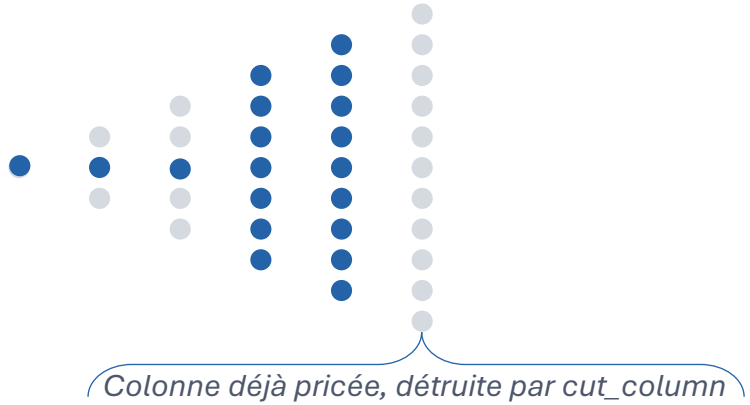
Tree.price() évalue l'option sur l'arbre intact. Pas de *cut_column* car le pic a déjà eu lieu et **l'arbre reste réutilisable** (grecques, prix européen).

0←T

BACKWARD - Génération et destruction

0→T

Tree.build_trunk() calcule probabilités et stocke sur le ***TrunkNode*** (pour *pruning*), puis ***cut_column()*** détruit la colonne pour bloquer l'explosion mémoire.



Tree._build_column() recrée 2 colonnes actives, applique le pruning, puis ***cut_column()*** détruit la colonne précédente ***O(N)***.

0←T

Le pruning en backward : l'astuce du masque binaire sur le tronc

Le problème : En backward, la probabilité d'atteinte ne se calcule qu'en avançant depuis la racine : il faut noter à l'aller quels nœuds sont élagués, puis le relire au retour, sans utilizer de listes ou de dictionnaires.

La solution : Encoder l'état dans un entier sur le noeud du tronc.
Chaque colonne est ancrée par son noeud central, le tronc *TrunkNode*. On y ajoute deux attributs scalaires : trun.pruned_up=0 et trun.pruned_down=0. Le j-ième noeud au-dessus du tronc correspond au bit d'indice (j-1).

À l'aller (Class *Tree*, méthode *_link_side*) :

```
mask |= 1 << (abs(j) - 1)
```

Allume le bit du noeud sans écraser les bits précédents.

Au retour (Class *Tree*, méthode *_was_pruned*) :

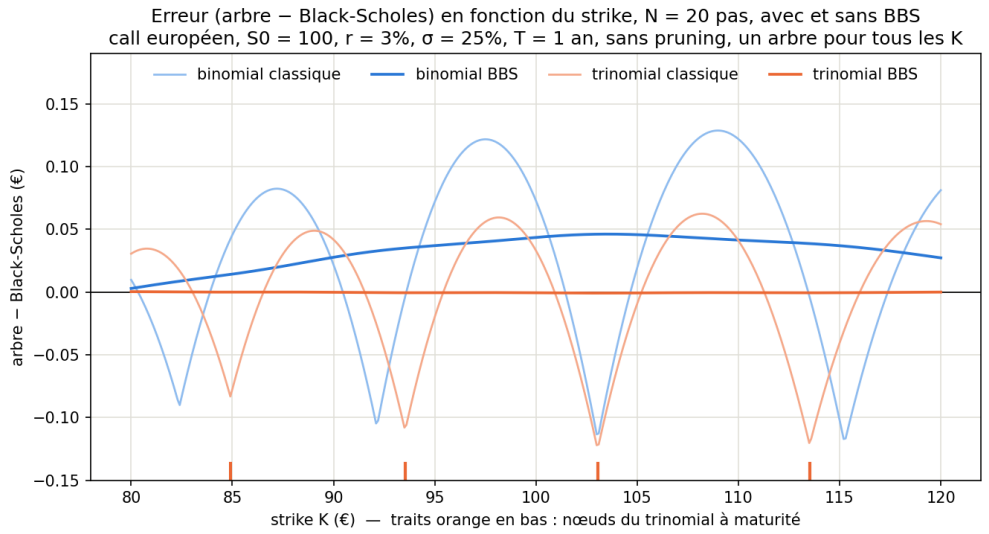
```
(trunk.pruned_up >> (j - 1)) & 1 == 1
```

Décaler les bits pour amener le bit cible sur la position 0, et l'isole avec un masque *&1*. Complexité en temps *O(1)*.



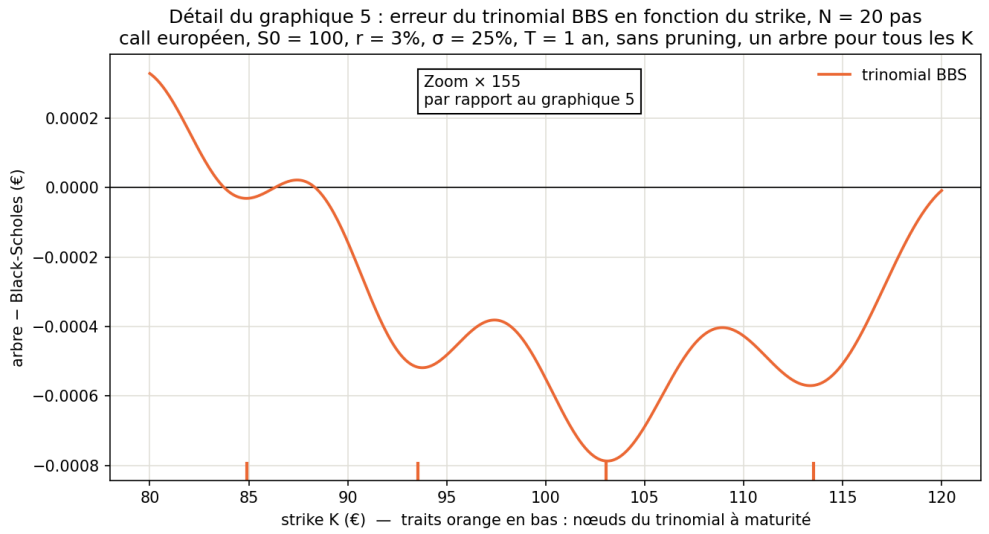
IV. MESURE DE L'ERREUR

BBS lisse l'erreur, le trinomial BBS l'efface



±0,13 €

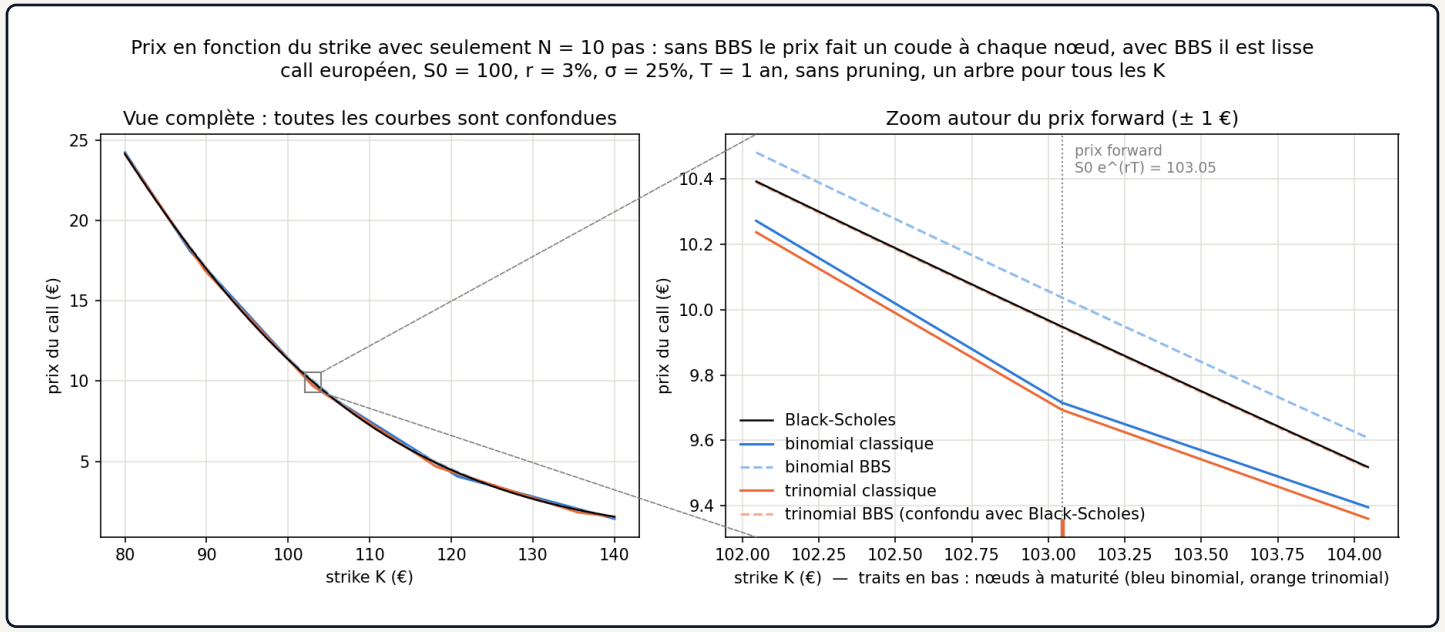
sans BBS, N = 20 : un creux chaque fois que K tombe sur un nœud



≈ 0

trinomial BBS : courbe lisse, il faut zoomer ×155 pour la voir

Pourquoi un creux quand K tombe sur un nœud



ce que change BBS

Au dernier pas, Black-Scholes compte **tous les prix autour de K**, pas seulement les nœuds : le coude du payoff est lissé.

La cassure tombe au forward (103,05 €), position du nœud final du tronc.

K tombe pile sur un nœud → les cas « juste au-dessus » y sont rangés → ce nœud paie exactement 0 → petits gains oubliés : prix trop bas

V. PRÉCISION VISÉE

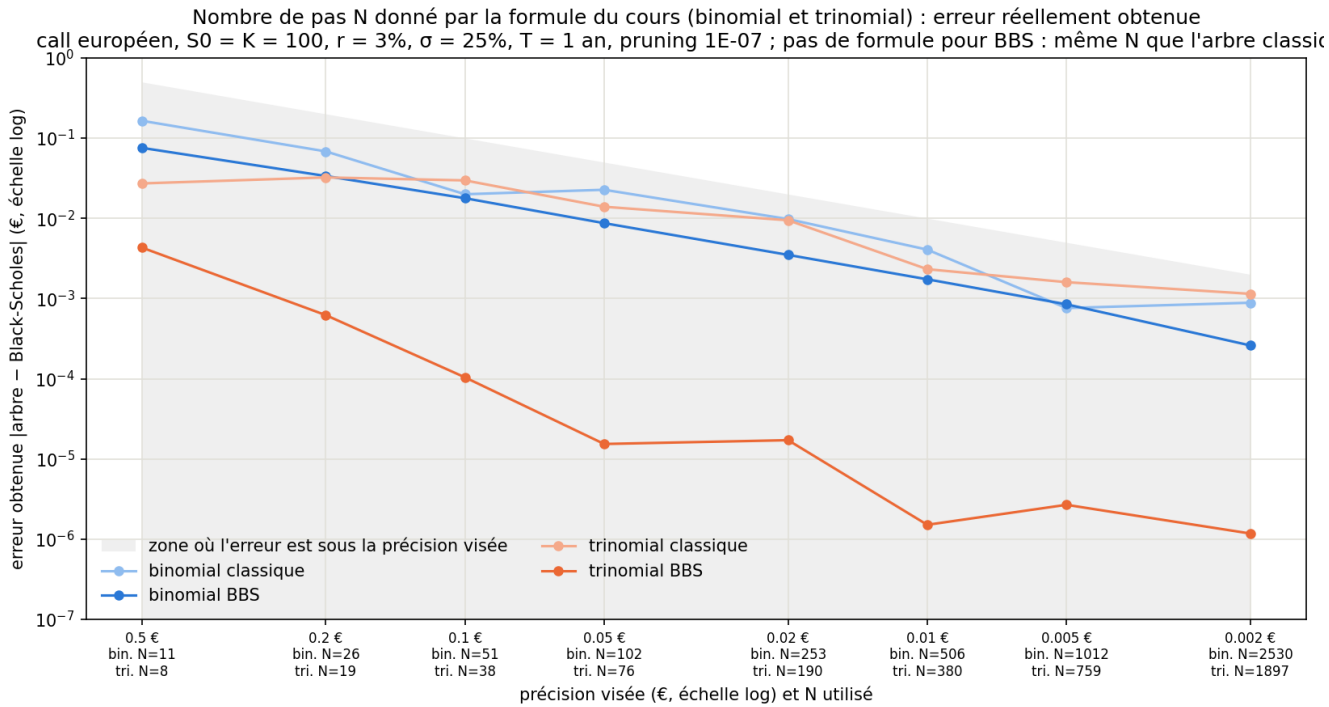
Choisir N pour une précision ε

`Model.from_precision`

$$N = \frac{c \cdot S_0 e^{rT} \cdot \sigma^2 T}{\sqrt{(2\pi) \cdot \varepsilon \cdot \sqrt{(e\text{^}(\sigma^2 T) - 1)}}}$$

c = 1/2 binomial

c = 3/8 trinomial



Pourquoi BBS marche si bien en trinomial

binomial · nœuds αS et S/α



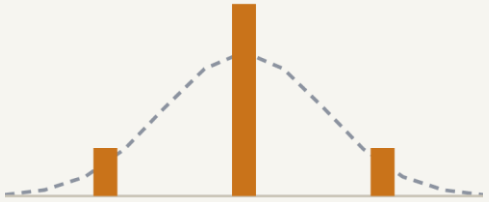
kurtosis 1

$p_1 + p_2 = 1$	somme
$p_1 \cdot \alpha S + p_2 \cdot S/\alpha = S\, e^{(r\Delta t)}$	espérance
$p_1 \cdot (\alpha S)^2 + p_2 \cdot (S/\alpha)^2 = S^2\, e^{(2r\Delta t)}\, e^{(\sigma^2 \Delta t)}$	variance

3 inconnues (p_1, p_2, α) · 3 équations

biais BBS : **+0,03 à +0,05 €**

trinomial · nœuds αF, F et F/α · F = S e^(rΔt)



kurtosis 3

$p_1 + p_2 + p_3 = 1$	somme
$p_1 \cdot \alpha F + p_2 \cdot F + p_3 \cdot F/\alpha = F$	espérance
$p_1 \cdot (\alpha F)^2 + p_2 \cdot F^2 + p_3 \cdot (F/\alpha)^2 = F^2\, e^{(\sigma^2 \Delta t)}$	variance
$p_1 \cdot (\alpha F)^4 + p_2 \cdot F^4 + p_3 \cdot (F/\alpha)^4 = F^4\, e^{(6\sigma^2 \Delta t)}$	ordre 4

4 inconnues (p_1, p_2, p_3, α) · 4 équations ⇒ α ≈ e^(σ√(3Δt))

erreur BBS : **< 0,001 €**

VI. BBS ET OPTION AMÉRICAINE

Pour une américaine, BBS n’apporte presque rien

put européen · trinomial BBS

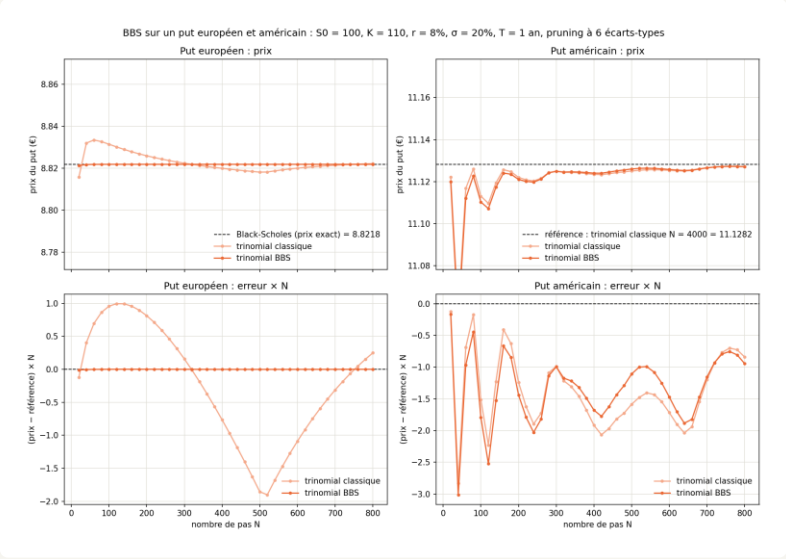
erreur × N ≈ 0

collé à Black-Scholes (8,8218 €) dès N = 20

put américain · trinomial BBS

erreur × N ∈ [−3 ; −0,2]

mêmes oscillations, sous la référence (11,1282 €)



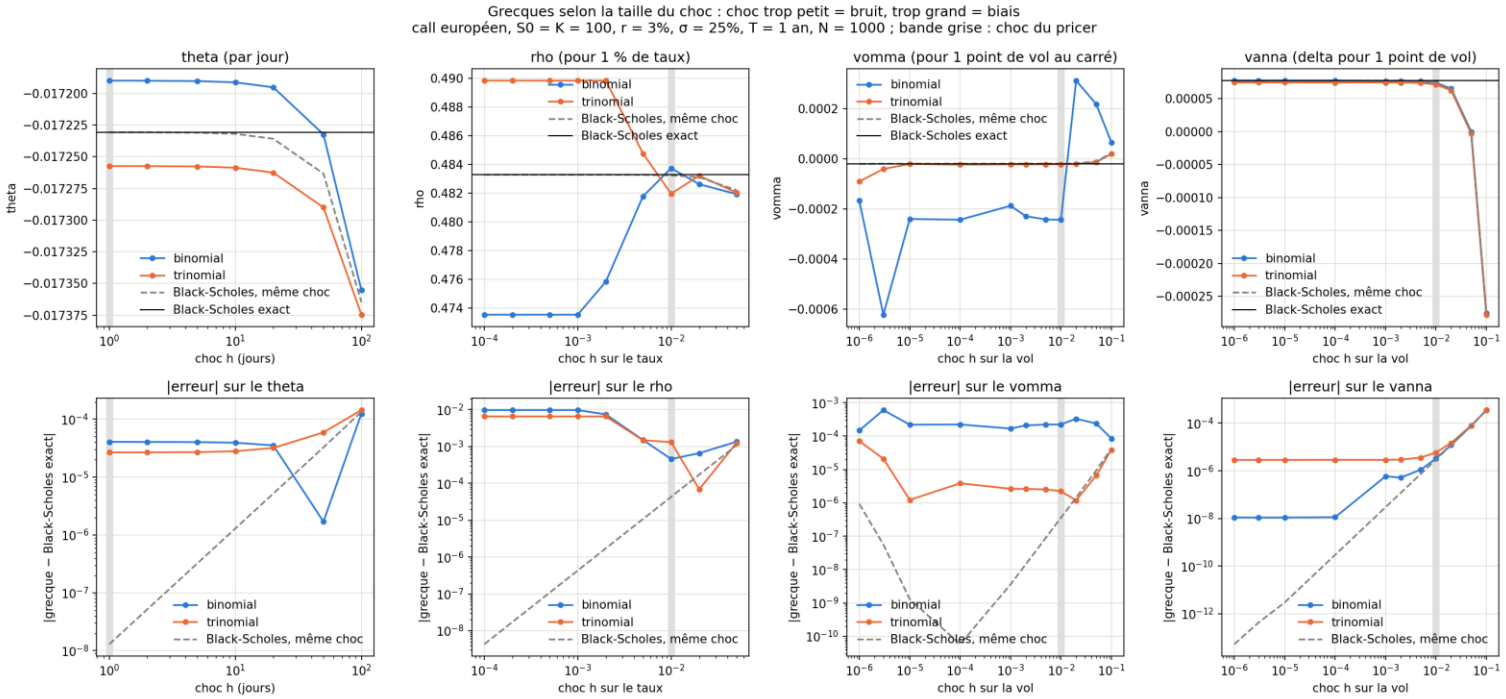
pourquoi : l’exercice anticipé

01 Le seuil d’exercice existe **à toutes les dates**, vu en escalier aux nœuds.

02 BBS ne corrige que **le dernier pas**.

03 L’arbre n’exerce qu’aux N dates de la grille : il **sous-estime** le prix.

La taille du choc : entre bruit et biais



Choc trop grand

Biais : la différence finie elle-même est fausse

Choc trop petit

Bruit : on mesure les irrégularités de l'arbre

Nos chocs

1 jour, 1 %, 1 point de vol : au creux de la courbe

Vomma : le binomial est faux d'un facteur 10, utiliser le trinomial

Un seul code pour toutes les différences finies

LA CLASSE

```
class OneDimDerivative:
    def __init__(self, function, other_parameters, shift=1.0):
        self.f, self.param, self.shift = function, other_parameters, shift

    def first(self, x):                # (f(x+h) - f(x-h)) / 2h
        up, down = self._around(x)     # f(x±h) gardées en mémoire
        return (up - down) / (2 * self.shift)

    def second(self, x, f_x):          # (f(x+h) - 2f(x) + f(x-h)) / h²
        up, down = self._around(x)
        return (up - 2 * f_x + down) / self.shift ** 2
```

LA FONCTION PASSÉE EN PARAMÈTRE

```
def price_vs_vol(params, vol):
    market, model, option, method = params
    m = Market(market.s0, market.r, vol, dividends=market.dividends)
    return price_tree(m, model, option, method,
                      grid_vol=market.vol)    # grille figée
```

Des fonctions en paramètres

On lui passe « prix selon σ, r ou la date » : vega, vomma, rho et thêta partagent le même code.

Valeurs réutilisées

f(x ± h) sont gardées : vega puis vomma ne reconstruisent aucun arbre de plus.

Grille figée pour la vomma

Seules les probabilités changent quand σ bouge. Sans cela, la vomma est très bruitée.

CONCLUSION

Il n’y a pas de réglage universel

Forward	la vitesse : 1,5 à 2× plus rapide
Backward	la mémoire : 70× moins, 1 Mo contre 80 Mo à N = 500
Pruning	temps ÷ 4 à 5, grâce au codage binaire : presque gratuit
Trinomial BBS	par défaut en européen : erreur < 0,001 €
Américaine	BBS n’aide pas : l’exercice anticipé est sous-estimé
Grecques	choc trop petit = bruit, trop grand = biais

Merci.